

# AI Cybersecurity Agent: Name, Strategy & Roadmap

We propose turning your Hermes fork into an **AI-powered security agent** tailored for Indian small businesses. We'll preserve the core agent code, skills, Docker setup and YAML configs (per your cleanup plan), while deleting unrelated docs/website files. For example, we keep `Dockerfile`, `docker-compose.yml`, all `*.yaml` configs, and the `skills/` directories, but remove translation docs (`README.es.md`, etc.), infographics, and non-code content.

## 1. Hermes Fork – Code Inventory

- **Kept:** Core agent logic (`agent/`, `cli.py`, `run_agent.py`), Docker files (`Dockerfile`, `docker-compose*.yaml`), all YAML configs (`*.yaml`, `.hadolint.yaml`), and the `skills/` and `optional-skills/` directories. These form the heart of the AI agent and must remain intact.
- **Deleted:** Nonessential docs and assets (e.g. extra `README.*.md`, `CONTRIBUTING.md`, the `website/`, `docs/`, `infographic/`, `.plans/`, and `LICENSE`). This declutters the repo for focus on security functionality.

*(No external citation needed – this matches your cleanup plan.)*

## 2. Competitive Landscape (SMB Security Agents)

Existing AI security tools focus on narrow domains. For example, **Twine Security** markets “AI Digital Employees” for identity management [20†L1-L9]. **Hex Security** runs continuous AI pentests on apps and infra [9†L112-L116]. **GhostEye** advertises the “first autonomous security loop for attacks on your people,” tracking attacker tactics and testing exposures [21†L238-L242]. **Cotool** provides an AI copilot that assembles context across tools and builds no-code automations for investigations [21†L331-L340]. Each of these addresses parts of the problem (identity, pentesting, incident triage), but none offers a unified, proactive agent acting on an entire organization graph.

Our product's edge will be this *digital-twin approach*: unifying an SMB's cloud accounts, code repos, networks, emails and personnel into one knowledge graph and reasoning over it. Rather than producing isolated alerts, our agent will plan and act (much like a CISO's assistant). In short, we combine the “**digital employee**” concept [20†L1-L9] with **organizational context** and autonomy that existing tools lack.

## 3. Integration & Data Sources

To build the security knowledge graph, we'll ingest data from all supported sources. Key data collection points include:

- **Cloud/Network APIs:** Pull infrastructure metadata via cloud APIs (e.g. Google Cloud Asset Inventory, AWS Organizations, Azure Resource Graph) [19†L342-L347]. This automatically

maps VMs, containers, IAM roles, domains, etc. into our graph. For on-prem or private clouds, we can run lightweight inventory scans (Nessus/Nmap or vendor APIs).

- **Dev & Code Repos:** Integrate with GitHub/GitLab APIs to track codebases, secrets, dependencies and deployment configurations. Automate scanning of new commits for leaked keys or insecure code.
- **Collaboration & Email:** Connect to Google Workspace (Gmail, Drive), Slack/Microsoft Teams, Jira/Trello, etc. via OAuth APIs to monitor tasks, alerts, and incident reports. For example, scanning Gmail or Slack for phishing reports or suspicious activity. (We will require user OAuth consent to access these, which is allowed on Google Cloud [48†L247-L252] .)
- **Endpoint & Network Telemetry:** Optionally use agent-based telemetry from endpoints/servers (endpoint security logs, EDR alerts, VPN logs, Windows domain, etc.). This rounds out the picture of user logins, lateral movement, and breach attempts.
- **Log & SIEM Data:** Ingest logs from SIEM or cloud logs (CloudTrail, CloudWatch, Stackdriver, etc.) for anomalous events.

Using these sources, we construct a Neo4j-style knowledge graph (a **digital twin of the SMB's IT environment** [19†L311-L319] ). The Neo4j team explains that a security graph may include “which systems connect to which systems, users and groups, permissions, critical assets” and events like user access and device connections [19†L311-L319] . This holistic graph enables our agent to answer questions like “What would happen if X account is compromised?” or “Which assets depend on this GitHub repo?” easily.

## 4. Unique Differentiators & India Fit

Key advantages over generic tools:

- **End-to-End Reasoning:** Unlike siloed scanners (secrets in code, vulnerability scanners, etc.), our agent correlates issues across domains. E.g., if a GitHub key leaks, the agent knows which AWS resources it touched and can trace risk to databases or customer data. This proactive “CISO assistant” model goes beyond mere alerts.
- **Autonomy & Planning:** The agent will automatically detect and act on risks. For example, in an ongoing incident it might: “rotate the leaked key, restart affected containers, patch exposed firewall rule, open tickets for affected teams, and draft a summary report.” This kind of autonomous response loop (similar to what GhostEye calls an “autonomous security loop” [21†L238-L242] ) differentiates us from static tools.
- **SMB Focus:** We target small/medium enterprises (<100 employees) that lack full-time security teams. Our UI and pricing will be far simpler than enterprise SOC products. Instead of complex dashboards, we'll offer a single portal (or even a mobile chatbot interface) for alerts and plans.
- **Indian Market Angle:** India has a huge MSME base – about 78.6 million registered SMBs as of Feb 2026 [38†L263-L266] – many underserved by enterprise security vendors. Most Indian SMBs operate mobile-first (India leads with ~854 million WhatsApp users [43†L209-L216] ). We can leverage this by providing mobile notifications or chat-based interfaces in English/Hindi. For example, sending SMS or WhatsApp alerts for critical incidents. We will also offer Hindi/local-language support to differentiate from Western tools.
- **SMB Security Needs:** Research shows Indian/SMB firms prioritize security but lag in execution [22†L170-L179] [22†L233-L239] . A Sage/IDC study notes 60% of SMBs plan to increase security spend, yet many lack proactive measures [22†L170-L179] . Our product aligns with this need by embedding security into daily ops, exactly as IDC recommends (“embed cybersecurity into AI initiatives and take an organization-wide approach” [22†L233-L239] ).

## 5. Implementation Roadmap (90-Day Hackathon)

We outline a 90-day plan to build an MVP that meets the XPRIZE requirements (Google Cloud + Gemini API). Key milestones:

1. **Day 0–10: Setup & Basic Integrations.** Fork Hermes code; configure Google Cloud (Cloud Run, Firestore) for deployment (XPRIZE requires a Google Cloud product [48†L247-L252] ). Implement Gemini API client and test a sample call (to satisfy the “use Gemini at least once” rule [48†L255-L259] ). Spin up Neo4j (or an in-memory graph DB) for the knowledge graph.
2. **Day 11–30: Data Ingestion.** Connect 1–2 data sources – e.g., GitHub (scan repos) and AWS (list resources) – and populate the graph. Build ETL pipelines to ingest this into the agent’s memory.
3. **Day 31–50: Core Agent Logic.** Develop agent “skills” for common security tasks. For example, a skill to detect a leaked secret and map its usage, or a skill to analyze IAM permissions from the graph. Integrate Gemini for advanced reasoning – e.g. summarizing an incident timeline or interpreting a security alert.
4. **Day 51–70: User Interface & Notifications.** Build a simple UI or chatbot. For instance, a web dashboard and a WhatsApp bot (via Twilio or WhatsApp Business API) to query the agent. Implement alerts – e.g. send email/SMS for critical findings. Ensure Gemini is used in at least one LLM call in the deployed app (required by rules [48†L255-L259] ).
5. **Day 71–90: Testing, Polish, Documentation.** Onboard one pilot customer (even a friendly local business) to test the workflow. Refine based on feedback. Prepare hackathon submission: record a demo video highlighting AI-driven workflows (in Small Business or Entrepreneurship category [48†L231-L239] ), and document system architecture. Ensure compliance (Gemini API, Google Cloud usage).

We’ll utilize Google Cloud credits (Cloud Run for deployment, BigQuery/Firestore for logs, etc.) per XPRIZE guidance [48†L247-L252] . The development stack remains Hermes (Python, Docker) plus graph DB (Neo4j or similar) and OAuth integrations.

## 6. Unique Product Name

After surveying the market, we choose a distinctive brand name. **Proposed Name:**

**“Rakshastra” (रक्षस्त्र).** This coined term fuses Sanskrit “Raksha” (protection) with “Shastra” (weapon/knowledge), evoking a “protective tool.”

- **Uniqueness:** A web search and industry review show no existing cybersecurity product named “Rakshastra.” It does not conflict with names like *Twine* (AI identity agent) [20†L1-L9] , *GhostEye* [21†L238-L242] , *AegisAI* [61†L1-L4] or *Cotool* [21†L331-L340] .
- **Brand Fit:** It conveys strength and an Indian identity (Sanskrit roots) while being easy to pronounce. It’s memorable and meaningful in context.
- **Justification:** Competitor names are generally English tech terms (e.g. “Hex Security,” “CloudGuard”). By contrast, “Rakshastra” signals both technology and cultural resonance. We will brand it simply as **Rakshastra AI** or **Rakshastra Security**.

In summary, our **“Rakshastra” agent** will deliver proactive, AI-driven security to Indian SMEs. It leverages the preserved Hermes agent framework, integrates with cloud/platform APIs to build a live digital twin [19†L311-L319] , and uses Gemini for intelligent reasoning. This aligns with XPRIZE requirements (AI-powered workflow in Small Business category, using Google Cloud & Gemini [48†L247-L252] [48†L255-L259] ) and addresses a real market need in India [22†L170-L179] [38†L263-L266] [43†L209-L216] .

**Sources:** Industry analyst reports on SMB cybersecurity [22†L170-L179] [22†L233-L239] , Neo4j on security knowledge graphs [19†L311-L319] [19†L342-L347] , and descriptions of related AI security agents [20†L1-L9] [21†L238-L242] [21†L331-L340] . These informed our strategy and naming.

---